

## Phase 1 CLI-Parsing

1. Check --help first. If set: print help text, terminate program immediately, no further checks needed.
2. Unknown/incorrect options from getopt\_long. Evaluate return value and abort on error.
3. For each option: type and value range check before anything is done with the value:
  - a. --cycles: valid uint32\_t? (no negative value, no overflow during parsing)
  - b. --latency-rom: valid uint32\_t
  - c. --rom-size: valid uint32\_t, must be 0 or a power of two
  - d. --block-size: valid uint32\_t
  - e. --rom-content: path specified? File exists? Read access available? (Only check file opening here, do not read it in yet)
  - f. --tf: if set, is the destination path writable? positional : path present? File exists and is readable?

## Phase 2 Reading the ROM Content File

1. Open file – if this fails despite the Phase 1 check (e.g., permissions have changed), abort cleanly.
2. Parse each line: is the value a valid number? Invalid line → error + abort.
3. Check number of read values against --rom-size. Since each value is 32 bits = 4 bytes, the maximum allowed number of values is  $\text{rom\_size} / 4$ .
  - a) More values than allowed → error, terminate program immediately.
  - b) Fewer values → no error, fill remaining values with 0.

## Phase 3 Reading the Request CSV

1. Are the number and order of fields correct? (Type,Address,Data,User,Wide)
2. Type field: only R or W allowed.
3. Address field: valid number (decimal/hex)?
4. Data field, depending on Type:
  - a) For R: must be empty, if a value is entered there --> error
  - b) For W: must be present and a valid number.
5. User field: valid number in uint8\_t range (0–255)?
6. Wide field: only T or F.
7. Combined check for write accesses with Wide = F: The value to be written must not be larger than 1 byte ( $> 0xFF$ ) --> error.

Each faulty line → output error message, terminate program (do not skip the line and continue, the specification requires aborting on a faulty input file).

## Phase 4 Simulation: per Request and Cycle

According to the specification, all requests within run\_simulation are already valid.

1. Routing check: Is addr within the ROM area ( $\text{addr} < \text{rom\_size}$ ) or the RAM area?
2. If ROM:
  - a. w set?  $\rightarrow$  immediate access error ( $\text{error}=1$ ).
  - b. r set?  $\rightarrow$  wait for latency, then  $\text{rdata} + \text{ready}=1$ ,  $\text{error}=0$ .
3. If RAM  $\rightarrow$  Permission check first:
  - a. Calculate block number from addr.
  - b. Query `getOwner(block)`: Is  $\text{user} == \text{owner}$  or owner unassigned or  $\text{user} \in \{0, 255\}$ ?
  - c. Attention edge case: For a 4-byte access ( $\text{wide}=T$ ), the 4 affected bytes can lie on a block boundary and thus be located in two different blocks with different owners. Specification requires authorization for "every byte being accessed".
  - d. No access right  $\rightarrow$  immediate error, no further main memory access.
4. If rights ok: Ownership update on write access:
  - a.  $\text{user} == 255 \rightarrow$  block is released (owner back to "unassigned").
  - b. otherwise (incl.  $\text{user} == 0$ )  $\rightarrow$  block is assigned to user.
5. Data Width Adaptation:
  - a.  $\text{wide}=T$ : direct 4-byte access via `mem_*`.
  - b.  $\text{wide}=F$ , Read: perform 4-byte access, wait for `mem_ready`, then extract the correct byte (Little-Endian!), rest of `rdata` to 0.
  - c.  $\text{wide}=F$ , Write: read first (Read-Modify-Write), wait for `mem_ready`, then replace the target byte in the read value, only then write back the combined 4-byte value, wait for `mem_ready` again.
6. `mem_ready` as a gate: For every single mem access (even for the two sub-accesses of the 1-byte write), the controller must actually wait until `mem_ready=1` before continuing.
7. Completion of the request: `ready=1`, error finally set accordingly (0 on success).

## Phase 5 End of Simulation / Output

1. Check termination condition of the simulation: Here we should clarify as a design decision (and document in the report) whether the simulation stops after exactly --cycles cycles, even if requests are still open, or whether it stops as soon as all requests have been processed.
2. Fill result struct: actually consumed cycles, total number of access errors.
3. Close tracefile if --tf was set.
4. Output the result in a readable format on the command line.

